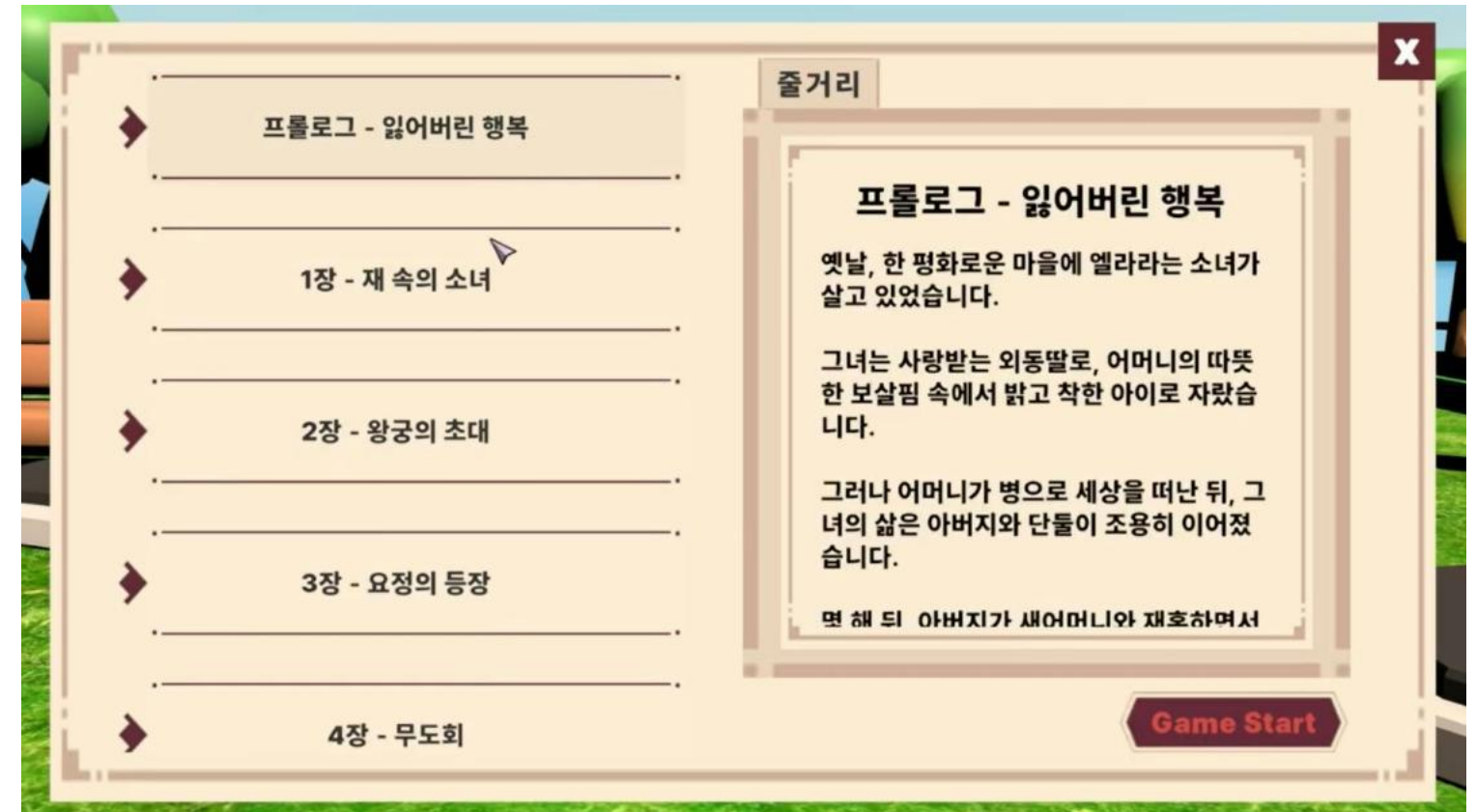


유니티 팀 프로젝트 - 대사 말하기 게임



기술 경험

- 유니티 생명주기 함수
- 코루틴
- 스프라이트 애니메이션
- 빌보딩
- Cinemachine

Tell The Story - 대사 말하기 게임

‘Tell The Story’ 는 플레이어가 직접 대사를 발음하여 NPC와 함께 연기를 이어가고, 그 과정에서 자신의 연기와 NPC들의 스토리를 확인하는 스토리텔링 시뮬레이션 게임입니다.

사용 기술 : Unity3D, JSON, NAVER CLOVA Speech(STT API)

<u>TTS 보이스</u>	<u>이미지 생성(타이틀 로고, 메인 게임 배경)</u>
볼리(VOLI)	- OpenAI의- Dall-E in Chat-GPT(프롬프트) - Microsoft Copilot(Edge Copilot)

장르 : 시뮬레이션

플랫폼 : Windows

엔진 버전 : Unity 2022.3.28f1

참여 인원 : 개발(2)

협업 도구 : Discord, Github

GitHub : [a-i-am/Unity3D-ActingGame-TellTheStory-](https://github.com/a-i-am/Unity3D-ActingGame-TellTheStory-)

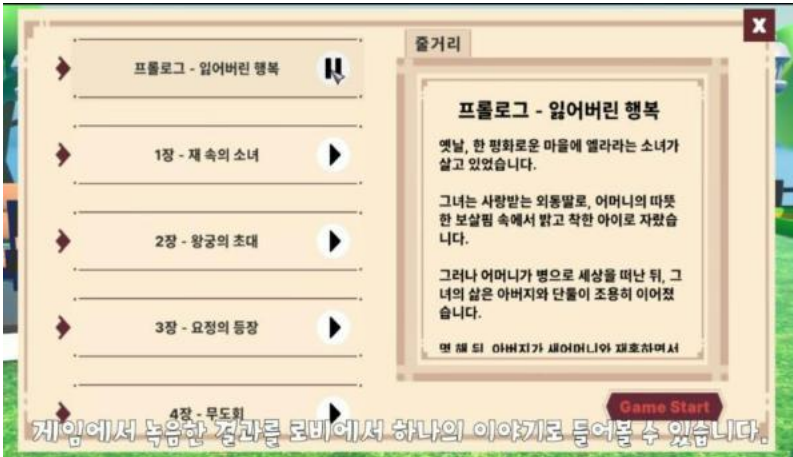
플레이 영상 : <https://youtu.be/nVsos3e-BUQ>

핵심 구현 기능

- 인 게임 로비 및 NPC 상호작용
- UI, 캐릭터 애니메이션 적용
- 유니티 에디터 커스텀 스크립트 적용

“NPC와 대사를 주고 받으며 이야기를 진행하는 게임”

1



로비 맵에서 함께 연기할 NPC 및 에피소드를 고릅니다.

2



자신의 연기 순서와 녹음 타이밍에 맞게 대사를 발음합니다.

플레이어 대사



STT(Speech To Text)
(녹음 후 TTS로 재생)

NPC 대사



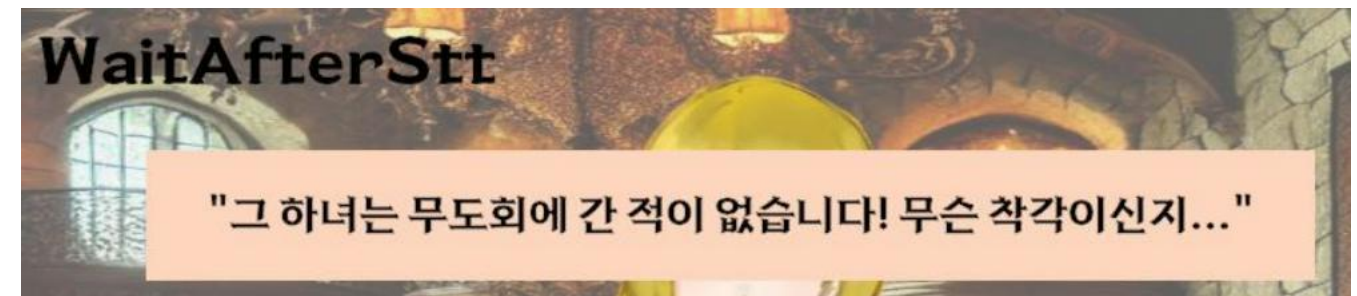
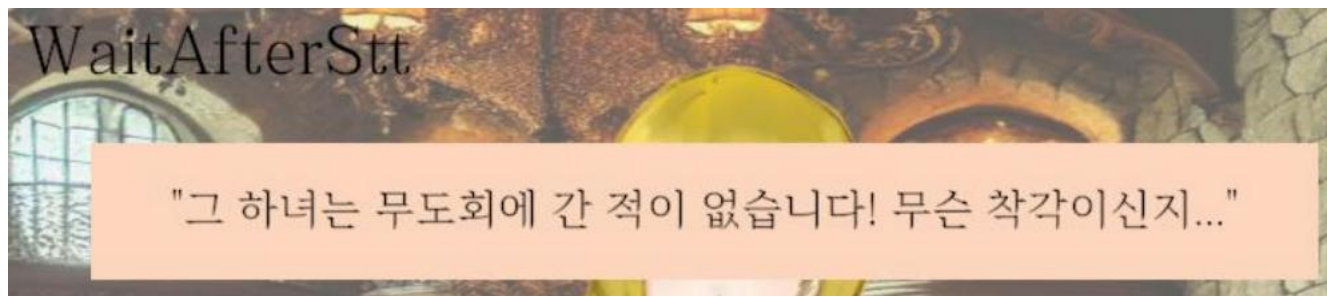
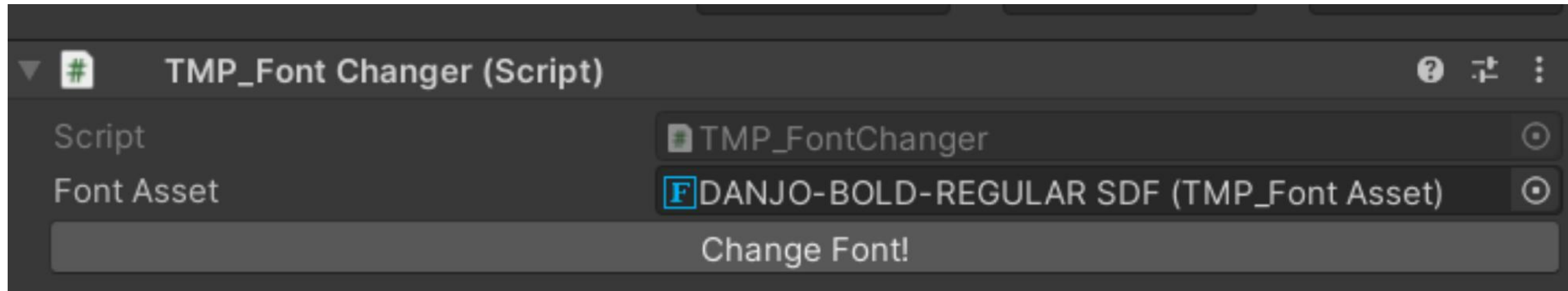
TTS(Text to Speech)
(NPC 목소리로 녹음된 TTS 재생)

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

에디터 내 UI 폰트 일괄 변경 기능

어울리는 폰트 비주얼을 빠르게 확인하기 위해, 에디터 내에서 버튼 클릭만으로 씬에 설정된 폰트를 특정 폰트로 일괄 변경할 수 있게 하였습니다.

그 덕분에 UI 등의 비주얼 작업에 들이는 시간을 줄여 게임 기능 완성 및 버그 수정에 보다 집중할 수 있었습니다.



(Github) TMP_FontChanger.cs

https://github.com/a-i-am/Unity3D-ActingGame-TellTheStory-/blob/main/Assets/Scripts/Font%20Changer/TMP_FontChanger.cs

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

8 Direction Sprite 작업을 위한 비주얼 에디터 적용

(Github) ActorBillboard.cs (210번줄 ~ 241번줄)

<https://github.com/a-i-am/Unity3D-ActingGame-TellTheStory-/blob/main/Assets/External%20Assets/EDSS/Scripts/ActorBillboard.cs>

캐릭터 방향 별 모션 적용에 유니티 Blend Tree를 사용 시 개체의 Position의 값 변화 상태(0, -1, 1 등)에 따라 스프라이트가 적용되게 구현해야 했습니다. 그래서 8 방향 애니메이션 스프라이트를 쉽게 관리하는 에셋 스크립트를 프로젝트에 맞게 수정하여 사용했습니다.

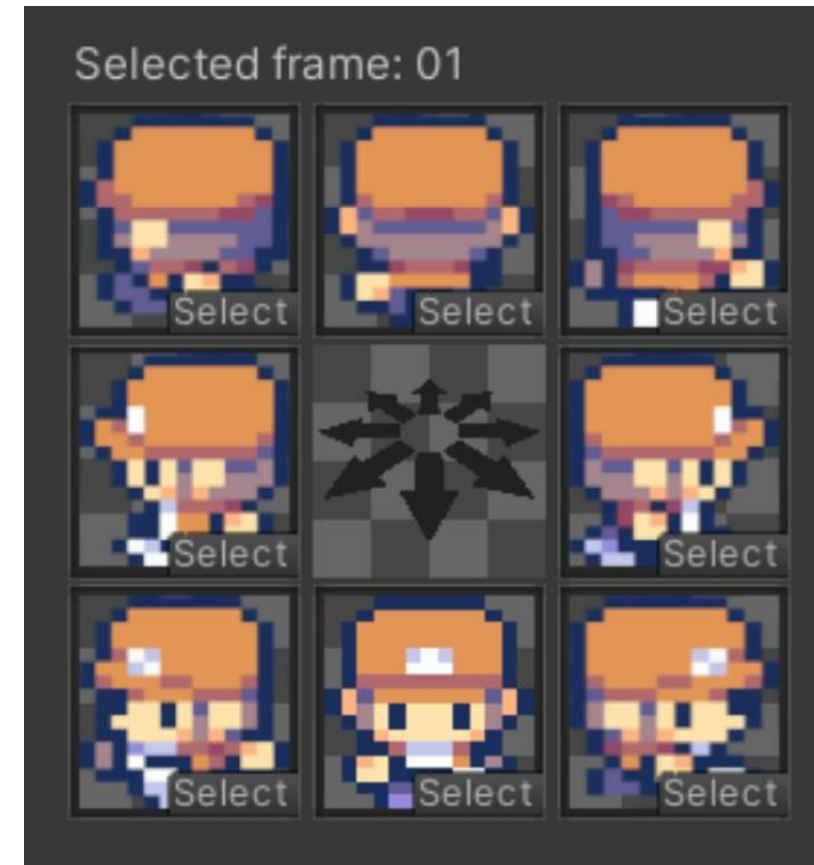
```
private void OnWillRenderObject()
{
    if (currentAnimation == null) ...
    if (spriteRenderer == null) ...
    bool isDialogueActive = DialogueManager.instance != null && DialogueManager.instance.isDialogueActive;
    beforeRenderBillboardEvent?.Invoke();
    Direction animDir = Direction.Down;
    if (currentAnimation.AnimType == ActorAnimation.AnimDirType.EightDirections && !isDialogueActive)
    {
        InputFlags flags = InputFlags.None;

        if (Input.GetKey(KeyCode.UpArrow) || Input.GetKey(KeyCode.W)) flags |= InputFlags.Up;
        if (Input.GetKey(KeyCode.DownArrow) || Input.GetKey(KeyCode.S)) flags |= InputFlags.Down;
        if (Input.GetKey(KeyCode.LeftArrow) || Input.GetKey(KeyCode.A)) flags |= InputFlags.Left;
        if (Input.GetKey(KeyCode.RightArrow) || Input.GetKey(KeyCode.D)) flags |= InputFlags.Right;

        animDir = directionTable[(int)flags];
    }

    animFrameDirection = (int)animDir;
    spriteRenderer.sprite = currentAnimation.GetSprite(currentFrameIndex, animFrameDirection);
}
```

OnWillRenderObject()는 해당 객체가 최소한 하나의 카메라에 보일 때만 호출되기 때문에, 화면 밖에 있는 객체에 대해서는 연산을 건너뛰어 CPU 부하를 줄일 수 있습니다.



- 사용자 입력에 따라 캐릭터 이동 스프라이트 애니메이션이 3D 공간에서 대각선 방향으로의 Position 변화대로 적용되도록 수정하였습니다.
- 그 결과, 플레이어는 자신이 누르는 이동 방향에 따라 2D 그래픽을 3D처럼 생각하고 움직일 수 있게 되었습니다.
- 또한 여러 플레이어의 비주얼을 언제든지 부담없이 교체하고 원하는 모션을 구현하기 편해져 작업 시간을 단축할 수 있었습니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

코루틴 활용 : 시간 지연 및 타이핑 효과 구현

(Github) PlayerController.cs

[Unity3D-ActingGame-TellTheStory-/Assets/Scripts/Lobby/Player/PlayerController.cs](#)
at main · a-i-am/Unity3D-ActingGame-TellTheStory-

코루틴을 활용하여 로비 내 NPC와의 대화 타이핑 효과를 구현하였습니다.

```
IEnumerator TypeSentence(DialogueLine dialogueLine)
{
    dialogueArea.text = "";
    foreach (char letter in dialogueLine.line.ToCharArray())
    {
        dialogueArea.text += letter;
        yield return new WaitForSecondsRealtime(typingSpeed);
    }
}
```

(96번줄 ~ 104번줄)

```
if (typingCoroutine != null)
{
    StopAllCoroutines();
}
```

```
public void DisplayNextDialogueLine()
{
    if (lines.Count == 0)
    {
        EndDialogue();
        return;
    }

    SoundManager.instance.PlayButton(0);

    DialogueLine currentLine = lines.Dequeue();

    characterIcon.sprite = currentLine.character.icon;
    characterName.text = currentLine.character.name;

    if (typingCoroutine != null)
    {
        StopCoroutine(typingCoroutine);
    }
}
```

(63번줄 ~ 80번줄)

- StopAllCoroutines() 는 모든 코루틴을 멈추기 때문에 불필요한 중단이 발생하거나 이전 코루틴과 새 코루틴이 동시에 실행될 수 있습니다.
- 특정 코루틴만 지정해서 중단할 수 있는 StopCoroutine()을 사용하여 타이핑

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

Enum & Switch Case : 동작 특성에 따른 호출 프레임 분리

(Github) PlayerController.cs

[Unity3D-ActingGame-TellTheStory-/Assets/Scripts/Lobby/Player/PlayerController.cs](#)
at main · a-i-am/Unity3D-ActingGame-TellTheStory-

로비 씬에서 플레이어 이동 상태와 NPC와 대화 중인 상태가 중첩되지 않게 해야 했기 때문에 Enum 으로 게임 내 상황 별 대화 상태를 구분 짓고, 각 상태에 의한 동작이 호출될 때 동시에 다른 동작이 간섭하지 않도록 Switch문으로 Case를 분리하였습니다.

```
enum PlayerState
{
    Moving,
    InDialogue
}
```

(4번줄 ~ 8번줄)

```
void Update()
{
    switch (currentState)
    {
        case PlayerState.Moving:
            HandleMovementInput();
            break;
        case PlayerState.InDialogue:
            HandleDialogue();
            break;
    }
}
```

(34번줄 ~ 54번줄)

```
void HandleMovementInput()
{
    float inputX = Input.GetAxisRaw("Horizontal");
    float inputY = Input.GetAxisRaw("Vertical");

    Vector3 moveDir = new Vector3(inputX, 0, inputY).normalized;
}
```

(83번줄 ~ 102번줄)

- Switch문이 어셈블리어로 변환될 때, 비연속적이면서 너무 case가 많으면 해시 테이블로 다뤄져서 탐색 비용이 증가하지만 해당 기능에서는 다룰 case가 적었기 때문에 적용하였습니다.

```
void FixedUpdate() (55번줄 ~ 59번줄)
{
    if (currentState != PlayerState.Moving) return;
    ApplyMovement();
}
```

```
void ApplyMovement() (113번줄 ~ 126번줄)
{
    Vector3 localMove = new Vector3(moveAmount.x, 0, moveAmount.z) * Time.fixedDeltaTime;
    rb.MovePosition(rb.position + localMove);
}
```

- 끊임 없이 빠르게 적용되어야 하는 동작은 Update()에 두고, 유니티 물리를 적용하는 로직은 FixedUpdate()에 두어 적절한 타이밍에 맞게 입력과 이동이 처리되도록 동작 호출 위치를 나누었습니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

그 외 개발 내역



3D 환경에서의 2D 스프라이트 *빌보딩

(Github) [PlayerController.cs](#)

[Unity3D-ActingGame-TellTheStory-/Assets/Scripts/Lobby/Player/SpriteBillboard.cs](#)
at main · a-i-am/Unity3D-ActingGame-TellTheStory-

*빌보딩(BillBoarding)

: 2D 오브젝트의 회전을 계속해서 변경하여 항상 카메라를 향하게 하는 것

유니티 Cinemachine 카메라 줌 인/줌 아웃

(Github) [CameraSetting.cs](#)

[Unity3D-ActingGame-TellTheStory-/Assets/Scripts/Lobby/Camera/CameraSetting.cs](#)
at main · a-i-am/Unity3D-ActingGame-TellTheStory-